中国海洋大学

计算机网络 课程实验报告

实验题目： wireshark 捕获数据包与协议分析

实验概述

【实验目的及要求】

1)了解抓包的基本原理:Wireshark 可以嗅探网络中的数据包,并解析各类网络协议。

2)学习使用 Wireshark 的基本操作: 启动抓包、设置网络接口、应用过滤规则等。

3)抓取不同种类的数据包。

4)训练和考察英文文献阅读的能力。

【实验环境】实验用的软硬件环境（配置）

1、硬件环境：

- 设备类型：个人计算机（PC）
- 操作系统：Microsoft Windows 10 家庭中文版
- 处理器：Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz，1992 Mhz，4 个内核，8 个逻辑处理器
- 存储：477 GB SSD WDC PC SN720 SDAPNTW-512G-1127
- 显卡：NVIDIA GeForce MX250 (2 GB), Intel(R) UHD Graphics 620 (128 MB)
- 网络适配器：Intel(R) Wireless-AC 9560 160MHz

2、软件环境

- 网络抓包工具：Wireshark 4.6.0 (v4.6.0-0-gcdfb6721e77c).（支持 TCP/UDP/ICMP 协议解析，捕获以太网 / Wi-Fi 数据包）

实验内容

【实验方案设计】

实验任务 1：了解抓包的基本原理:Wireshark 可以嗅探网络中的数据包,并解析各类网络协议。

1.1 抓包的核心前提：网络数据的 “可捕获性”

网络中的数据包并非默认“可被任意抓取”，其可捕获性取决于网络拓扑和网卡工作模式，这是抓包的基础。

1.1.1 网络拓扑的影响

不同网络结构下，数据包的传播范围不同，直接决定 Wireshark 能抓取到哪些数据：

网络类型	数据传播特点	抓包范围	典型场景
共享式网络（如早期集线器组网）	数据包“广播”到所有端口，所有设备都能接收	可抓取整个网络内的所有数据包	老式以太网（已少见）
交换式网络（如交换机组网）	数据包通过“MAC 地址表”定向转发到目标端口，不广播	默认仅能抓取“发送给本机”和“从本机发出”的数据包	家庭 / 实验室局域网（主流）
无线网络（Wi-Fi）	数据包在无线信道中广播，需解密才能解析	可抓取信道内所有数据包，但加密包需密钥解密	手机 / 电脑连同一 Wi-Fi

1.1.2 网卡的“混杂模式”（promiscuous mode）

默认情况下，网卡处于“正常模式”：仅接收目标 MAC 地址为自身或广播 / 组播地址的数据包，其他数据包会被网卡硬件过滤，不传递给操作系统。而 Wireshark 抓包的核心第一步，是将网卡切换到混杂模式：

- 作用：让网卡 “不过滤” 任何经过的数据包，无论目标 MAC 是否为自身，都将数据包传递给操作系统，供 Wireshark 读取。
- 注意：混杂模式仅能突破 “网卡硬件过滤”，无法突破交换机的 “定向转发”（交换式网络中，若需抓其他设备的数据包，需配置交换机镜像端口或用无线抓包）。

1.2 Wireshark 抓包的完整流程

Wireshark 本身不直接“接收 数据包，而是依赖操作系统的网络接口和抓包驱动，按“数据捕获→数据过滤→数据存储”的流程工作，具体步骤如下：

1.2.1 步骤 1：依赖抓包驱动，从网卡获取原始数据包

Wireshark 无法直接与硬件交互，需通过 底层抓包驱动对接操作系统的网络接口，获取原始数据包（未经过协议解析的二进制数据）。

1.2.2 步骤 2（可选）：基于过滤规则，筛选目标数据包

若直接捕获所有数据包，会产生大量冗余数据（如无关的广播包、其他协议的包）。Wireshark 支持抓包前过滤和抓包后过滤，减少数据量。

●抓包前过滤（Capture Filter）：在 “开始抓包” 前设置规则（如仅抓 TCP 协议、仅抓目标 IP 为 192.168.1.100 的包），不符合规则的数据包直接被丢弃，不进入后续流程；

●抓包后过滤（Display Filter）：抓包后对已捕获的数据包按规则筛选（如 tcp.port == 80 仅显示 HTTP 流量），不改变原始捕获数据，仅控制显示内容。

1.2.3 步骤 3：存储原始数据包到文件（.pcap 格式）

Wireshark 会将捕获的原始数据包（二进制格式）存储为.pcap 或.pcapng 格式文件（后者支持更多元数据，如时间戳、网卡信息），方便后续分析或复现。文件中每个数据包都包含：*时间戳*（捕获时的系统时间）、*数据包长度*（实际传输的字节数）、*原始二进制数据*（未解析的底层数据）。

1.3 Wireshark 解析协议的核心：基于网络分层模型“拆解”数据包

抓包只是第一步，Wireshark 的核心价值是解析各类网络协议——将二进制的原始数据包，按 TCP/IP 分层模型（或 OSI 模型）逐层“拆解”，转化为人类可理解的明文信息。

1.3.1 协议解析的逻辑：从“底层到上层”逐层解析

TCP/IP 模型分为 “链路层→网络层→传输层→应用层”，Wireshark 按此顺序逆向解析（先解底层头部，再解上层数据）.

分层（TCP/IP 模型）	协议字段（Wireshark 解析结果）	解析逻辑
链路层（以太网）	源 MAC 地址（如 00:11:22:33:44:55）、目标 MAC 地址（如 66:77:88:99:AA:BB）、帧类型（如 0x0800 表示 IP 协议）	先读取以太网帧头部（前 14 字节），提取 MAC 地址和后续协议类型，确定下一层是 IP 协议。
网络层（IP）	源 IP（如 192.168.1.10）、目标 IP（如 www.baidu.com 对应的 180.101.49.12）、协议号（如 0x06 表示 TCP 协议）	读取 IP 头部（通常 20 字节），提取 IP 地址和传输层协议类型，确定下一层是 TCP 协议。
传输层（TCP）	源端口（如 54321）、目标端口（如 80，HTTP 默认端口）、序列号（如 123456）、确认号（如 789012）	读取 TCP 头部（通常 20 字节），提取端口号和连接控制字段，确定应用层是 HTTP 协议。

应用层 （HTTP）	请求方法（如 `GET`）、请求 URL（如 `/index.html`）、请求头（如 `Host: www.baidu.com`）、响应内容	读取 TCP 数据段中的“应用层数据”，按 HTTP 协议格式解析请求 / 响应字段，最终显示明文。

1.3.2 协议解析的依据：内置 “协议解析库”

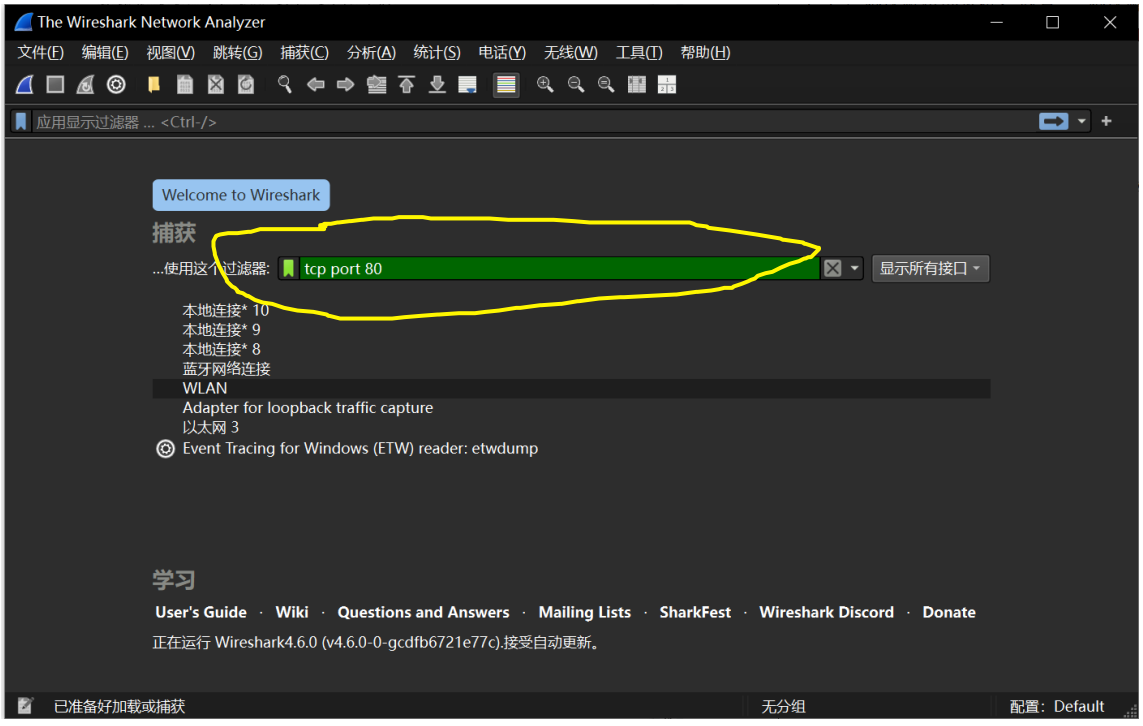
Wireshark 能解析上千种协议（如 TCP、UDP、ICMP、HTTP、DNS、SSL/TLS 等），核心是内置了协议解析库。

实验任务 2：学习使用 Wireshark 的基本操作：启动抓包、设置网络接口、应用过滤规则等

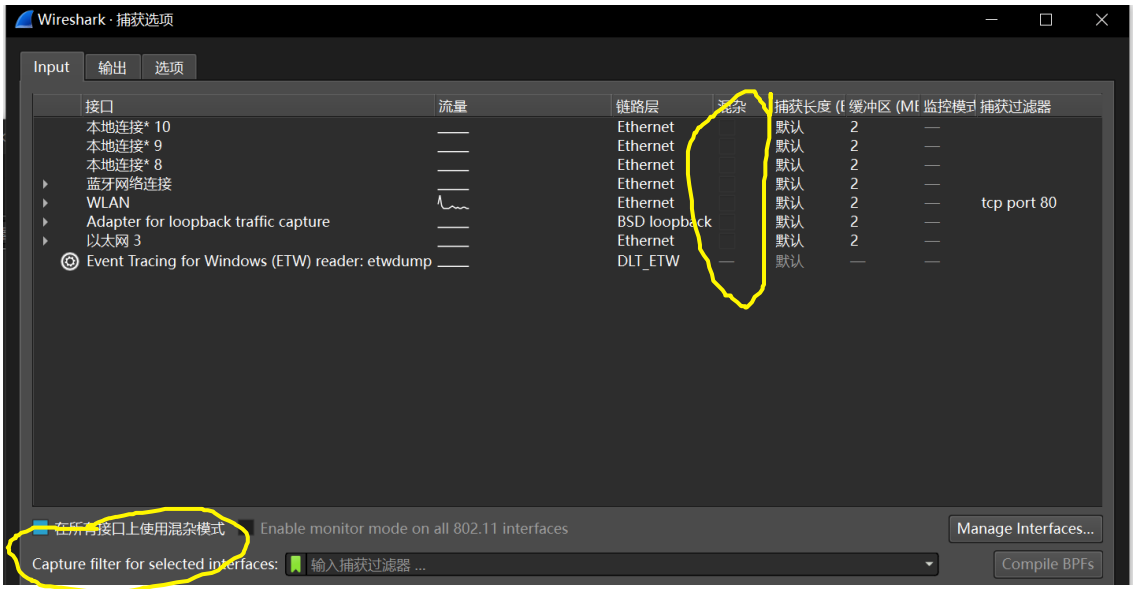
2.1 启动抓包、设置网络接口、应用过滤规则等

- 主页面如下图所示，在过滤器中输入“tcp port 80”，该规则仅记录标准 Web 流量，避免捕获计算机发送的其他类型数据包。

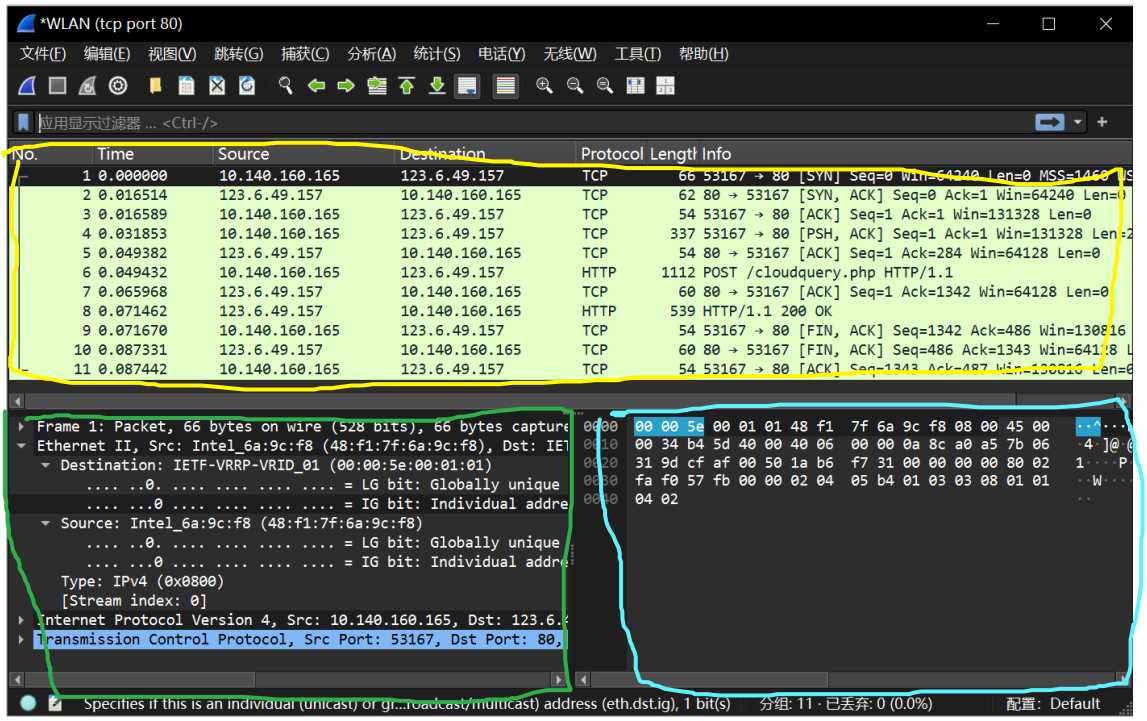
（未找到“enable network name resolution”（启用网络名称解析），此设置可将发送和接收数据包的计算机地址转换为名称，便于识别数据包是否往返于本机）



- 在捕获选项中取消混杂模式选项



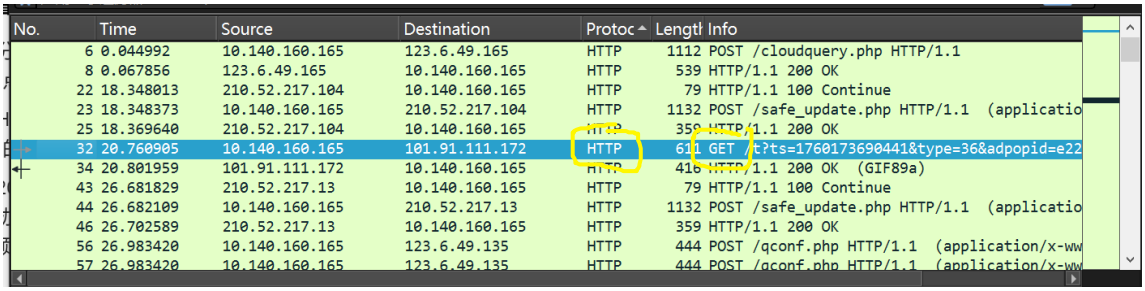
- 选定要捕获的接口，双击启动捕获，出现页面如下图。一共有三部分页面，中间面板显示所有捕获的数据包列表（序列、时间戳、源 IP、终点地址 ip、使用协议等），左下方面板可查看选中的数据包的协议层结构，右下面板显示数据包对应的字节内容。



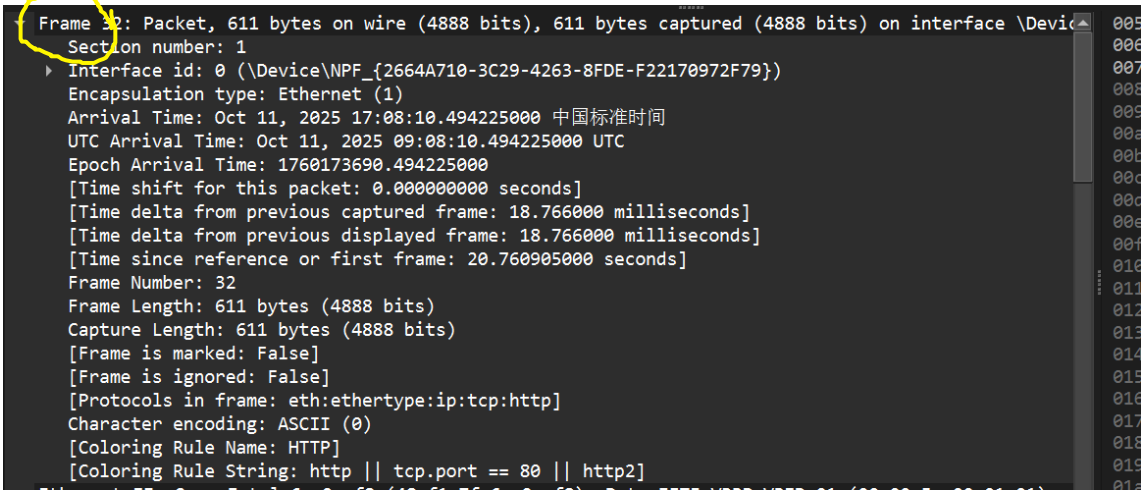
实验任务 3：抓取不同种类的数据包

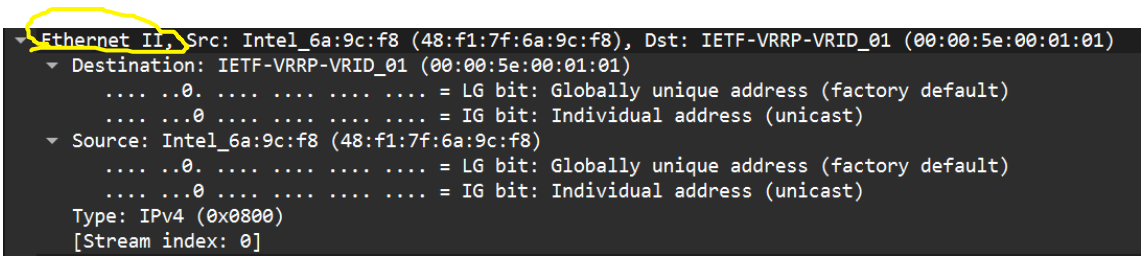
- 选择协议列为“HTTP”且信息列为“GET”的数据包（可点击列标题排序查找），该数据包为计算机向服务器发送的 Web（HTTP）请求包，可查看其

从链路层到应用层的协议嵌套结构。具体来说，HTTP 作为应用层协议用于访问 URL。与大多数互联网应用类似，它运行在 TCP/IP 传输层和网络层协议之上。链路层和物理层协议取决于网络环境，但通常会根据设备类型进行组合：有线设备通常采用以太网协议，无线设备则使用 802.11 协议。

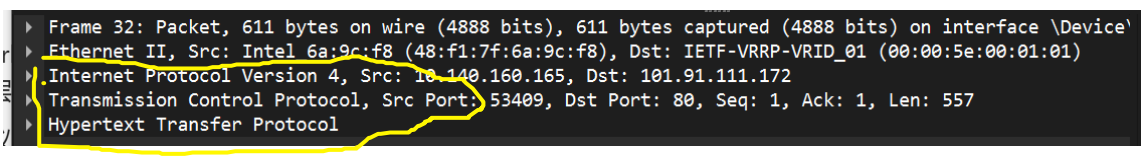


Wireshark 的首个数据块是“帧”。这并非协议本身，而是记录数据包整体信息的字段，包含捕获时间、长度等关键参数。第二个数据块是“以太网。需要注意的是，即便在计算机上使用 802.11 协议进行流量分析，仍可能看到以太网数据块而非 802.11 数据块，原因在于我们要求 Wireshark 在捕获选项中以以太网格式传输数据，因此系统将真实的 802.11 数据包头转换成了伪以太网格式。

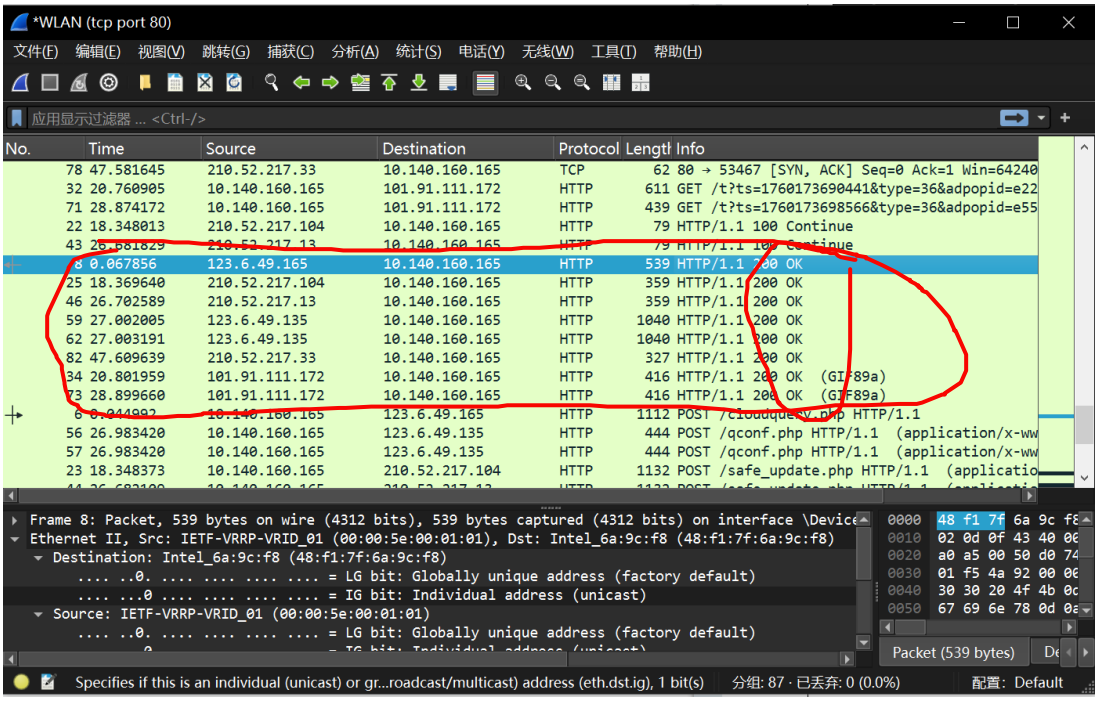




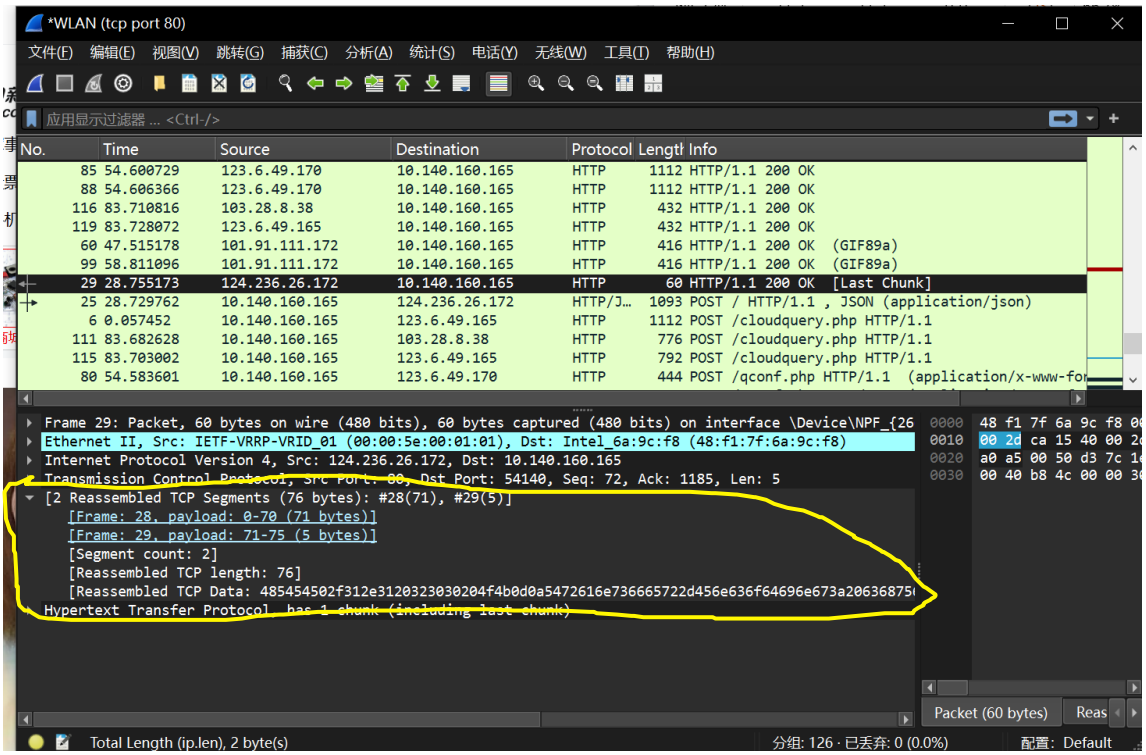
接下来是 IP、TCP 和 HTTP 协议，它们的排列顺序正是我们所期望的。请注意，这里的顺序是从协议栈底层到顶层递进的。这是因为当数据包沿协议栈向下传输时，底层协议的头部信息会被添加到上层协议信息的前端。也就是说，在传输过程中，底层协议会优先出现在数据包的“线上传输”中。



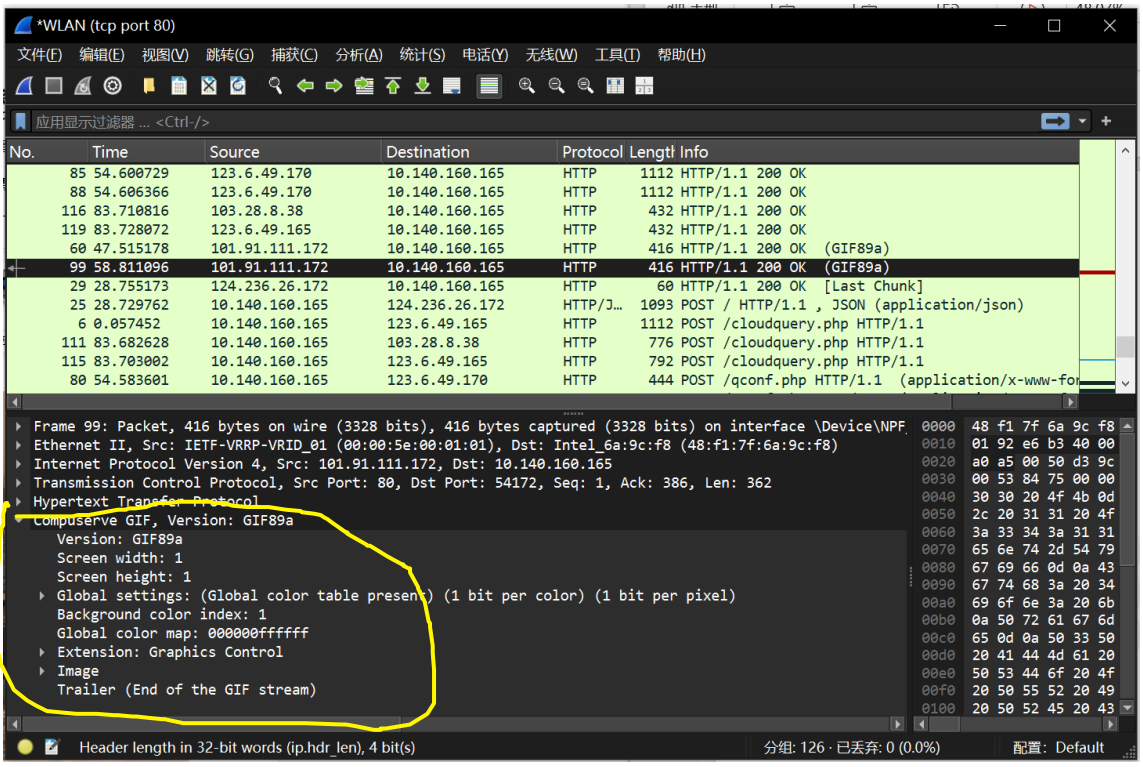
●查找信息列含“200 OK”的 HTTP 数据包，该数据包为服务器向计算机发送的响应包，对比其与 HTTP GET 包在协议块结构上的差异



第一个额外数据块显示“[2 Reassembled TCP Segments...]”。具体细节可能因网络捕获结果而异，但该数据块描述的远不止单个数据包本身。网页响应通常以一系列数据包形式在网络上传输，这些数据包在抵达计算机后会被重新组合。标记为 HTTP 的数据包是整个网页响应的最后一部分，该数据块列出了构成完整响应的各个数据包。尽管这些数据包仅传输 HTTP 响应的部分内容，但每个都标有 TCP 协议。只有当所有数据包组合起来才能完整呈现 HTTP 消息时，最终才会显示 HTTP 协议的数据包，并列出这些组合成完整响应的数据包。



第二个附加数据块显示“Compuserve GIF，version:GIF89a”。虽然具体细节会因情况而异，但这个区块描述的是被获取网页的内容类型。在本例中，其类型为 image/gif，但也可能是 text/xml、text/html 等多种格式。与帧记录类似，这并非真正的网络协议，而是 Wireshark 生成的数据包内容描述，用于帮助我们理解网络流量。

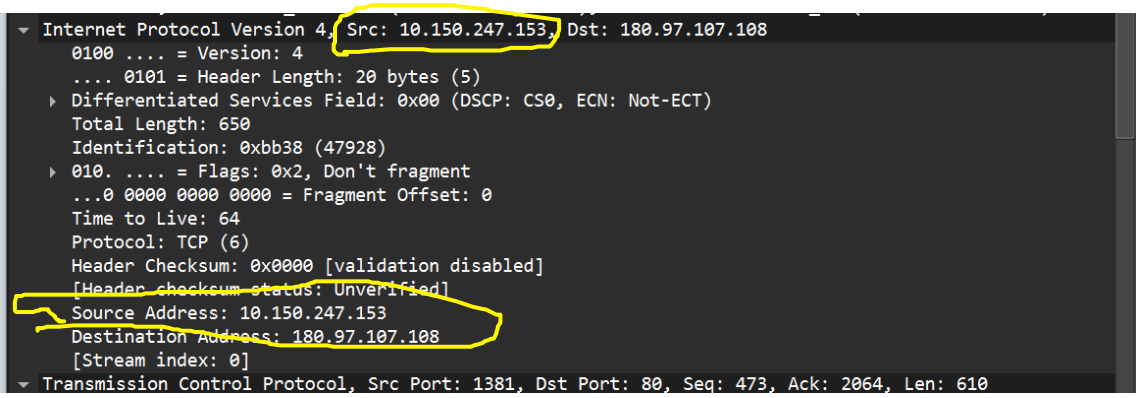


实验任务 4：packet Structure

Turn-in: Hand in your packet screenshot. Find out the IP address of your computer. Find out the protocol name of Application Layer and guess the packet are from which application that your are running now.

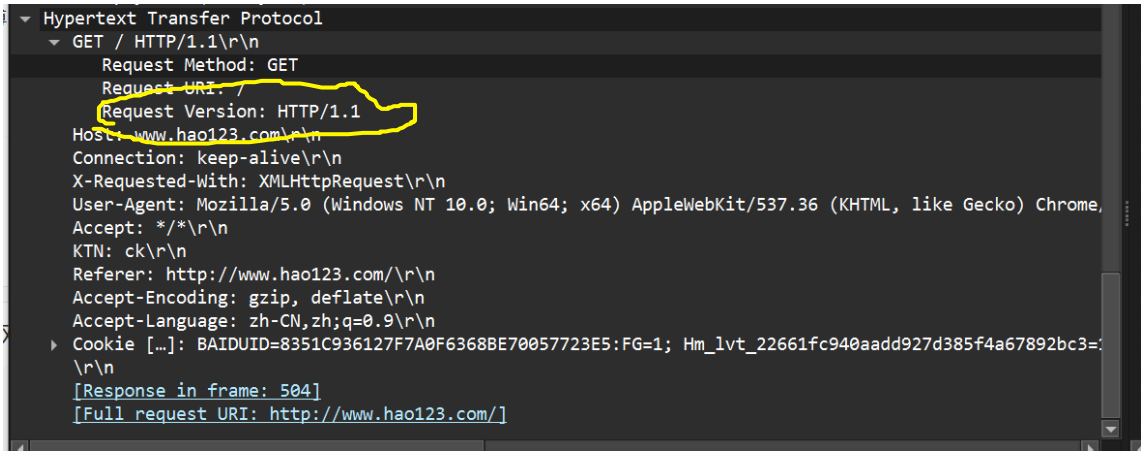
- Find out the IP address of your computer.

  ip：10.150.247.153



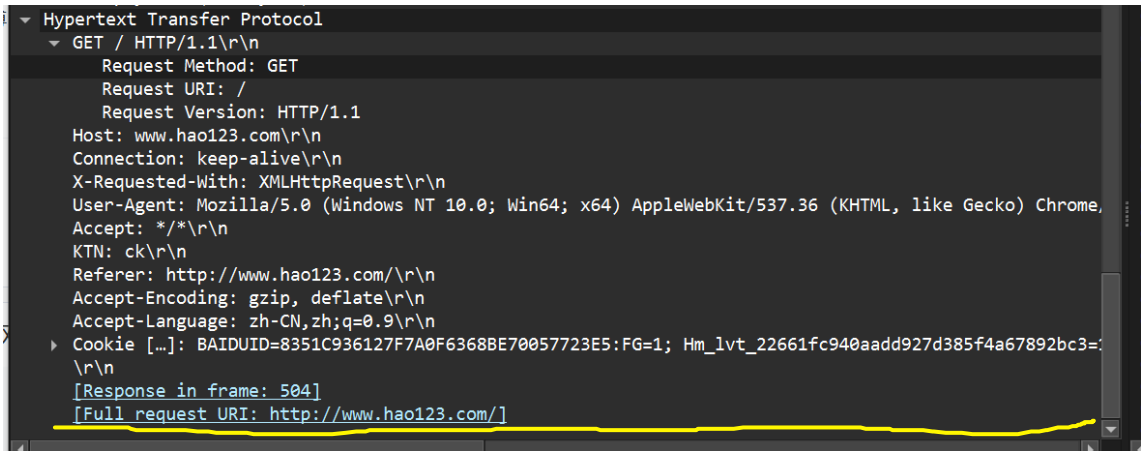
- Find out the protocol name of Application Layer

  name：HTTP /1.1



- guess the packet are from which application that your are running now.

http://www.hao123.com/



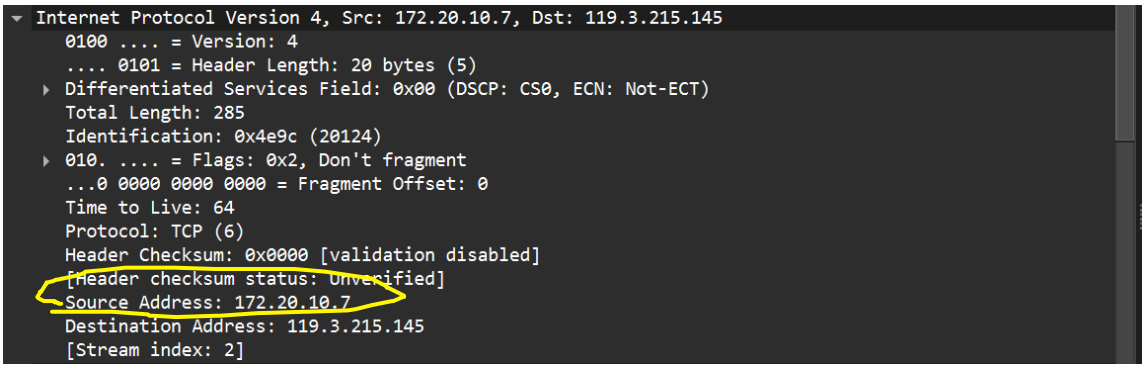
实验任务 5：Capture Packets from Other students

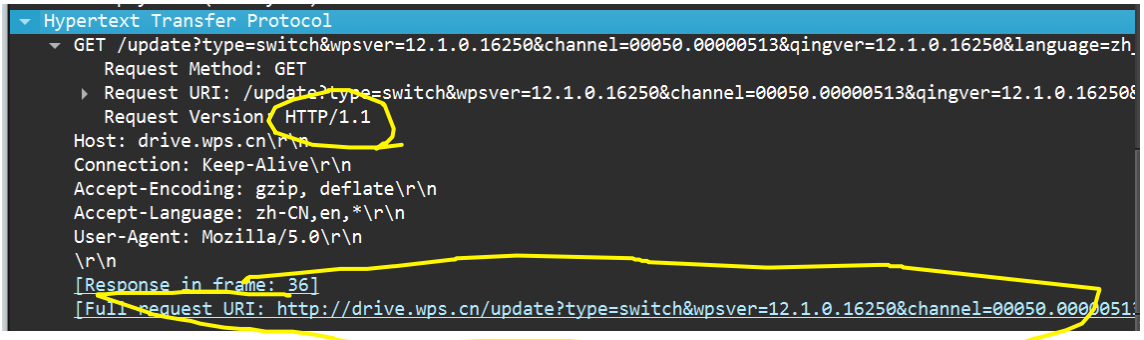
Turn-in: Hand in this new packet screenshot. Find out the IP address of the computer. Find out the protocol name of Application Layer and guess the packet are from which application that your roommate is running now.

ip:172.20.10.7

protocol name:HTTP/1.1

application:WPS





小结

本次实验学习了如何使用 wireshark 捕获数据包，了解了 wireshark 的工作原理以及页面功能，且可以通过过滤器在捕获前或者捕获后对所获得的数据包进行删减，方便获取想要的信息。

同时，学习了通过混杂模式获取非本机的数据包，前提是需要两台机器在同一个局域网之上，例如链接同一个网络热点，从而通过 IP 地址判断非本机的数据包。